

Student Course & Grade Management System

Project Report

VITyarthi - Build Your Own Project

Field	Details
Student Name	Shivansh pandey
Registration Number	26BCE11235
Course Name	Introduction to problem solving
Course Code	1021
Faculty	Vaishnavi S
GitHub Repository	https://github.com/anshuthehero11-stack/EduRecord_vityarthi-project
Date	28-09-2026

Table of Contents

1. Introduction
2. Problem Statement
3. Functional Requirements
4. Non-functional Requirements
5. System Architecture
6. Design Diagrams
7. Design Decisions and Rationale
8. Implementation Details
9. Screenshots / Results
10. Testing Approach
11. Challenges Faced
12. Learnings and Key Takeaways
13. Future Enhancements
14. References

1. Introduction

This project is a Student Course & Grade Management System written in Python. It is a console program which works with a simple numbered menu. The user can add students, enrol them in courses, save grades and check which students have failed.

I chose this project because keeping student records is something that happens in every school and college, and it is easy to understand. It also let me use the things we learnt in the course such as classes, inheritance, sets, dictionaries, loops and conditions in one small but complete program.

2. Problem Statement

In small classes and coaching groups, student details, courses and marks are often written in notebooks or different Excel sheets. Because of this it takes time to find one student's record, a student can accidentally be enrolled in the same course twice, and it is difficult to quickly see who has failed.

The aim is to make a simple program which keeps students, their courses and their grades in one place and gives quick answers to these questions.

Objectives

- To store student details (ID and name) in an organised way
- To enrol students in courses without duplicates
- To allow grades only for courses in which the student is enrolled
- To show the failed students (grade below 50) quickly
- To search any student by ID and show the courses and grades
- To apply OOP concepts (classes, inheritance) and data structures (set, dictionary)

Scope The program works in the console and stores data in memory only. There is no database, file saving, login or GUI in this version.

3. Functional Requirements

The project has three major functional modules, and each has a clear input and output.

Module	Feature	Input	Output
A. Student management	Add student	Student ID, name	'student added'
A. Student management	Search student by ID	Student ID	Name, list of courses with grades (or 'no

			grade' / 'no courses'), or 'no student found'
B. Course enrolment	Enrol student in a course	Student ID, course name	'course enrolled', or 'already enrolled', or 'no student found'
C. Grades and results	Add grade	Student ID, course name, grade	'grade saved', or 'not enrolled in course', or 'no student found'
C. Grades and results	Show failed students	None	Name, course and marks of each grade below 50, or 'no failed students'
Menu	Exit	Option 6	Program ends

User workflow The user starts the program, and the menu is shown. The user types a number, gives the inputs asked by the program, and the result is printed. Then the menu is shown again. This repeats until the user types 6.

4. Non-functional Requirements

Requirement	How it is handled in this project
Usability	Simple numbered menu, and every input has a clear prompt (for example 'student id', 'course name'). Messages are short and easy to understand.
Reliability / Error handling	The program checks whether the student exists before enrolling, grading or searching. It blocks duplicate enrolment and grades for courses that are not enrolled, and shows 'wrong input try again' for a wrong menu option. (Known gap: a non-numeric grade is not handled yet, see Challenges.)

Performance / Resource efficiency	Students are stored in a dictionary, so finding a student by ID is a direct lookup. Courses are stored in a set, so checking a duplicate course is quick. Data is small and kept in memory, so the program uses very little memory.
Maintainability	The logic for a student is inside classes (pr and st), separate from the menu loop. Inheritance is used so that common data is written only once.
Portability	Only standard Python 3 is used, with no external libraries, so it runs on any system that has Python.

5. System Architecture

The program is divided into three layers. The interface layer is the menu loop that takes input and prints output. The logic layer is made of the classes which contain the methods for enrolling and grading. The data layer is a dictionary that stores all student objects in memory.

(Reference: Figure 1: System architecture)

6. Design Diagrams

6.1 Use Case Diagram

(Reference: Figure 2: Use case diagram)

- User (teacher/coordinator) actions: Add student, Enroll student in course, Add grade, Show failed students, Search student by ID, Exit program.
- Options 2, 3 and 5 first check that the student ID exists.

6.2 Workflow Diagram

(Reference: Figure 3: Workflow of the menu program)

- Start -> Print menu options -> Read choice -> Execute 1 of 6 choices (or 'wrong input') -> Print result -> Loop back to menu.

6.3 Sequence Diagram (Add Grade)

(Reference: Figure 4: Sequence diagram for adding a grade)

- User chooses option 3 -> Enters ID, course, grade.

- Program checks if ID exists in dictionary d.
- Calls gr(course, grade) on student object.
- Checks if course in course set cs.
- Updates grades dictionary gs[course] = grade.

6.4 Class Diagram

(Reference: Figure 5: Class diagram)

- pr (person): + i : student id, + n : student name, + mk1(i, n)
- st (student) inherits pr: + cs : set of courses, + gs : dict {course: grade}, + mk2(i, n), + ad(c) : 1 or 0, + gr(c, g) : 1 or 0
- Main program stores objects in dictionary d.

6.5 ER Diagram

Not applicable. This project does not use a database or file storage. All data is kept in a Python dictionary while the program is running.

7. Design Decisions and Rationale

Decision	Reason
Dictionary for students (key = student ID)	Searching by ID is fast and easy, and there is no need to loop through all students.
Set for courses of a student	A set does not allow duplicates, so enrolling in the same course twice is detected easily.
Dictionary for grades (course: grade)	Each course has one grade, and the course name can be used directly to get or update it.
Base class pr and child class st	pr keeps the common data (id, name) and st adds courses and grades. This shows inheritance.
Methods return 1 or 0	The menu part can know if the operation worked and only then print 'course enrolled' or 'grade saved'.
Grades allowed only for enrolled courses	It keeps the data correct, since a student cannot have marks in a course they never took.
Pass mark fixed at 50	It is the usual pass mark, and it is kept

	simple. It can be made changeable later.
Console menu instead of GUI	It keeps the focus on logic and concepts and it is enough for the subject.

8. Implementation Details

The project is written in Python 3 in one file, VITYARTHI_Project.py. It has two classes and a main menu loop.

- **class pr:** method mk1(i, n) saves the student's id and name.
- **class st(pr):** inherits from pr. mk2(i,n) calls mk1 and creates an empty set cs (courses) and an empty dictionary gs (grades).
 - ad (c) adds course c to the set. If it is already there it prints 'already enrolled' and returns 0, otherwise it returns 1.
 - gr(c, g) saves grade g for course c only if c is in the set. Otherwise it prints 'not enrolled in course' and returns 0.
- **Main loop:** a dictionary d stores the students. A while loop prints the menu, reads the choice and uses if/elif to run the selected option.
- **Failed students (option 4):** loops over every student and every grade and prints the ones where the grade is below 50. A flag variable is used to print 'no failed students' if none is found.
- **Search (option 5):** prints the name and then each enrolled course with its grade, or 'no grade' if the grade is not added yet.

Grades are converted to float when they are entered, so decimal marks like 45.5 are also accepted.

9. Screenshots / Results

```

43     x=input("course name ")
44     s=d[i].add(x)
45     if s==1:
46         print("course enrolled")
47     else:
48         print("no student found")
49 elif c==3:
50     id=input("student id ")
51     if i in d:
52         x=input("course name ")
53         g=float(input("grade "))
54         s=d[i].gr(x,g)
55         if s==1:
56             print("grade saved")
57         else:
58             print("no student found")
59 elif c==4:
60     for k in d:
61         for x in d[k].gs:
62             if d[k].gs[x]<50:
63                 print("student",d[k].n,"failed",x,"with",d[k].gs[x])
64             else:
65                 pass
66         if k==d:
67             print("no failed students")
68 elif c==5:
69     id=input("student id to search ")
70     if i in d:
71         print("name",d[i].n)
72         if len(d[i].cs)!=0:
73             print("no courses")
74         for x in d[i].cs:
75             if x in d[i].gs:
76                 print("course",x,"grade",d[i].gs[x])
77             else:
78                 print("course",x,"no grade")
79     else:
80         print("no student found")
81 elif c==6:
82     pass
83 else:
84     print("wrong input try again")

```


10. Testing Approach

I did manual testing: I ran the program, gave different inputs and compared the output with what I expected. I did not write automated unit tests for this version. Both normal cases and wrong inputs were tried.

#	Test case	Expected	Actual	Result
1	Add student 101, Rahul	student added	student added	Pass
2	Enrol 101 in Maths	course enrolled	course enrolled	Pass
3	Enrol 101 in Maths again	already enrolled	already enrolled	Pass
4	Enrol/ grade with unknown id	no student found	no student found	Pass
5	Grade for course not enrolled	not enrolled in course	not enrolled in course	Pass
6	Grade 72 for enrolled course	grade saved	grade saved	Pass
7	Grade 38, then option 4	student listed as failed	listed with 38.0	Pass
8	Grade 49.9, then option 4	listed as failed	listed with 49.9	Pass
9	Grade exactly 50, then option 4	not listed (pass)	no failed students	Pass
10	Option 4 with no students	no failed students	no failed students	Pass
11	Search valid id with no	name and 'no courses'	name and 'no courses'	Pass

	courses			
12	Search wrong id	no student found	no student found	Pass
13	Menu option 9	wrong input try again	wrong input try again	Pass
14	Add student with an id that already exists	should be blocked	old record is replaced (courses lost)	Fail (limitation)
15	Enter 'abc' as grade	error message	program crashes with ValueError	Fail (limitation)

11. Challenges Faced

- **Choosing data structures:** at first I was not sure how to store courses and grades together. I decided to use a set for courses and a dictionary for grades, and this solved the duplicate enrolment problem easily.
- **Keeping the grade valid:** I had to stop grades from being added for courses the student is not enrolled in, so I made the method check the course set first.
- **Knowing if a method worked:** the menu should print 'saved' only if it really worked, so the methods return 1 or 0.
- **Input problems found in testing:** while testing I found that typing letters as a grade crashes the program, and that adding the same student ID again replaces the old record. I have not fixed these in this version, and they are listed in future enhancements.
- **Code style:** I used very short names (pr, st, d, mk1). It works, but I realised it is hard to read later, so better names would be good for the next version.

12. Learnings and Key Takeaways

- How to use classes and inheritance in a real program instead of only in examples
- Why a set and a dictionary are useful, a set for unique values and a dictionary for fast lookup by key
- Testing is important because even a small program had bugs I had not noticed until I tried wrong inputs
- Input validation should be planned from the beginning, not added at the end
- Planning the workflow first (menu, input, output) makes writing the code easier

13. Future Enhancements

- Handle wrong input with try/except (for example a non-numeric grade) and check that grades are between 0 and 100
- Do not allow adding a student whose ID already exists
- Save data in a file (CSV/JSON) or SQLite so it is not lost when the program closes
- Split the code into separate files (models, menu, storage) and rename variables to clearer names
- Add automated unit tests
- Add options to edit or delete a student or a grade, and to calculate GPA / percentage
- Make a GUI or a web version

14. References

- Python Software Foundation, The Python Tutorial: <https://docs.python.org/3/tutorial/>
- Python documentation, Classes (inheritance): <https://docs.python.org/3/tutorial/classes.html>
- Python documentation, Data Structures (sets and dictionaries): <https://docs.python.org/3/tutorial/datastructures.html>
- VITyarthi course material and the Build Your Own Project guidelines document